UNIVERSIDAD ESTATAL AMAZÓNICA
Unidad de Organización Curricular: Básica
Campo de Estudio: Básica

GUÍA DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Período Académico: 2025-2026	**Nº Práctica:** 04
Asignatura: Árboles y grafos	**Cód. Asignatura:** UEA-L-UFB-032
Nombre de la Unidad: Árboles y grafos	**Semana de práctica:** 13-14-15-16
Ambiente: Escritorio y Espacios abiertos	**Duración (horas):** 8
Título de la práctica	Guía de Prácticas #04: Implementación y representación de árboles y grafos.
Tipo de práctica	Prácticas experimentales de escritorio ■ Prácticas experimentales en laboratorio □ Prácticas experimentales en espacios abiertos □
Introducción	Las estructuras de datos constituyen un pilar fundamental en el desarrollo de software, ya que permiten organizar, almacenar y manipular información de manera eficiente. Entre estas, los árboles binarios destacan por su capacidad de representar relaciones jerárquicas y optimizar procesos de búsqueda y recorrido (Cormen et al., 2009). En este contexto, el presente trabajo aborda la implementación de un árbol binario utilizando el lenguaje de programación Python. El uso de árboles binarios permite modelar múltiples problemas computacionales, facilitando operaciones como inserción, búsqueda y recorrido de datos. Estas estructuras son ampliamente utilizadas en áreas como bases de datos, compiladores y sistemas de archivos (Goodrich & Tamassia, 2015). Por ello, su comprensión resulta esencial en la formación académica en el área de programación. En este práctico experimental se desarrolla una solución basada en la creación de nodos enlazados y el uso de recursividad para recorrer la estructura en preorden, permitiendo evidenciar el comportamiento y funcionamiento de los árboles binarios en un entorno real de programación (Weiss, 2014).

Fundamentación Teórica

. Un árbol binario es una estructura de datos no lineal en la que cada nodo puede tener como máximo dos hijos, denominados hijo izquierdo e hijo derecho. Esta estructura permite representar datos de forma jerárquica, siendo ampliamente utilizada en algoritmos de búsqueda y clasificación (Sedgewick & Wayne, 2011).

La recursividad es una técnica de programación en la cual una función se llama a sí misma para resolver subproblemas más pequeños. En el caso de los árboles, esta técnica es especialmente útil para realizar recorridos como preorden, inorden y postorden (Knuth, 1998).

El recorrido en preorden consiste en visitar primero el nodo raíz, luego el subárbol izquierdo y finalmente el subárbol derecho. Este tipo de recorrido es utilizado para copiar estructuras de árboles o evaluar expresiones matemáticas representadas en forma jerárquica (Cormen et al., 2009)

Un árbol está compuesto por:

- Nodo raíz
- Nodos hijos
- Nodos hoja
- Subárboles

Uno de los tipos más comunes es el árbol binario, en el cual cada nodo tiene como máximo dos hijos (izquierdo y derecho).

Los recorridos más utilizados en árboles son:

- Preorden
- Inorden
- Postorden

Estos recorridos permiten acceder a todos los nodos del árbol de manera ordenada, siendo esenciales en la manipulación de datos.

Antecedentes

El estudio de estructuras de datos ha evolucionado junto con el desarrollo de la informática, siendo los árboles una de las estructuras más antiguas y utilizadas en la computación. Desde sus primeras aplicaciones en la teoría de grafos, han sido adaptados para resolver problemas complejos en distintos campos tecnológicos (Knuth, 1998).

Diversos estudios han demostrado la eficiencia de los árboles binarios en comparación con estructuras lineales, especialmente en operaciones de búsqueda y ordenamiento, donde se logra reducir significativamente el tiempo de ejecución (Goodrich & Tamassia, 2015).

Actualmente, los árboles binarios son fundamentales en la implementación de estructuras avanzadas como árboles AVL, árboles B y árboles de búsqueda binaria, utilizados en sistemas modernos de almacenamiento y procesamiento de datos (Sedgewick & Wayne, 2011).

Objetivos

Objetivo general

Implementar y analizar una estructura de datos tipo árbol binario utilizando Python, aplicando técnicas de programación orientada a objetos y recursividad.

Objetivos Específicos

Comprender la estructura y funcionamiento de los árboles binarios.

Desarrollar un programa que permita crear y enlazar nodos.

Aplicar un recorrido recursivo en preorden.

Analizar los resultados obtenidos durante la ejecución del código.

Contextualización

El presente práctico se desarrolla en el contexto académico de la asignatura Estructura de Datos, donde se busca fortalecer los conocimientos relacionados con estructuras dinámicas y algoritmos de recorrido. El uso del lenguaje Python facilita la implementación debido a su sintaxis clara y versátil (Lutz, 2013).

La práctica permite simular escenarios reales donde se requiere organizar información jerárquica, como sistemas de archivos, bases de datos o árboles de decisión, lo cual evidencia la aplicabilidad de este tipo de estructuras en el entorno profesional

Justificación

El desarrollo de esta práctica se justifica en la necesidad de fortalecer los conocimientos sobre estructuras de datos no lineales, específicamente los

	árboles, los cuales son fundamentales en la programación y el desarrollo de software. Permite adquirir habilidades prácticas en la implementación de algoritmos, así como comprender la importancia de la eficiencia en el manejo de datos. Este conocimiento será de gran utilidad en su formación profesional y futura inserción laboral. El estudio de los árboles binarios es fundamental en la formación de profesionales en el área de sistemas, ya que proporciona herramientas para resolver problemas complejos de manera eficiente. Su implementación permite desarrollar habilidades en lógica de programación y pensamiento estructurado (Weiss, 2014). Además, el uso de recursividad en este tipo de estructuras fortalece la capacidad de análisis y optimización de algoritmos, lo cual es clave en el desarrollo de software de alto rendimiento. Por ello, este práctico contribuye significativamente al aprendizaje práctico del estudiante.

Desarrollo	La práctica se desarrolló en un entorno de escritorio, utilizando un computador con acceso a internet y herramientas de programación como Python. **Paso 1: Elaboración de árboles** Se diseñaron dos ejemplos de árboles para representar estructuras jerárquicas.

1. Árbol 1

```
    A
   / \
  B   C
 / \   \
D   E   F
```

Árbol 1

El primer árbol binario implementado presenta una estructura jerárquica donde el nodo raíz contiene el valor principal a partir del cual se derivan los demás nodos. Cada nodo posee como máximo dos hijos, organizados en subárbol izquierdo y subárbol derecho, lo que permite representar la información de forma estructurada.

En este caso, los valores han sido distribuidos de manera que se puede observar claramente la relación padre-hijo entre los nodos, facilitando la comprensión de la estructura. Este árbol sirve como ejemplo básico de construcción manual de un árbol binario, donde se establecen explícitamente las conexiones entre nodos.

Además, sobre este árbol se aplica un recorrido en preorden, lo que permite verificar el correcto funcionamiento de la estructura y del algoritmo recursivo implementado en el programa.

2.

```
Árbol 2
   10
  / \
 5   20
/ \  /
3  7 15
```

Árbol 2

El segundo árbol binario corresponde a una variación de la estructura inicial, en la cual se modifican los valores y la organización de los nodos con el objetivo de demostrar la flexibilidad de esta estructura de datos. A pesar de los cambios, se mantiene la propiedad fundamental de los árboles binarios, donde cada nodo tiene como máximo dos hijos.

Este segundo ejemplo permite evidenciar cómo diferentes configuraciones pueden afectar la forma del árbol, ya sea haciéndolo más balanceado o desbalanceado, lo cual influye directamente en el rendimiento de los algoritmos aplicados.

Al igual que en el primer caso, se realiza un recorrido en preorden para validar la correcta construcción del árbol y comprobar que la función recursiva opera adecuadamente en distintas estructuras

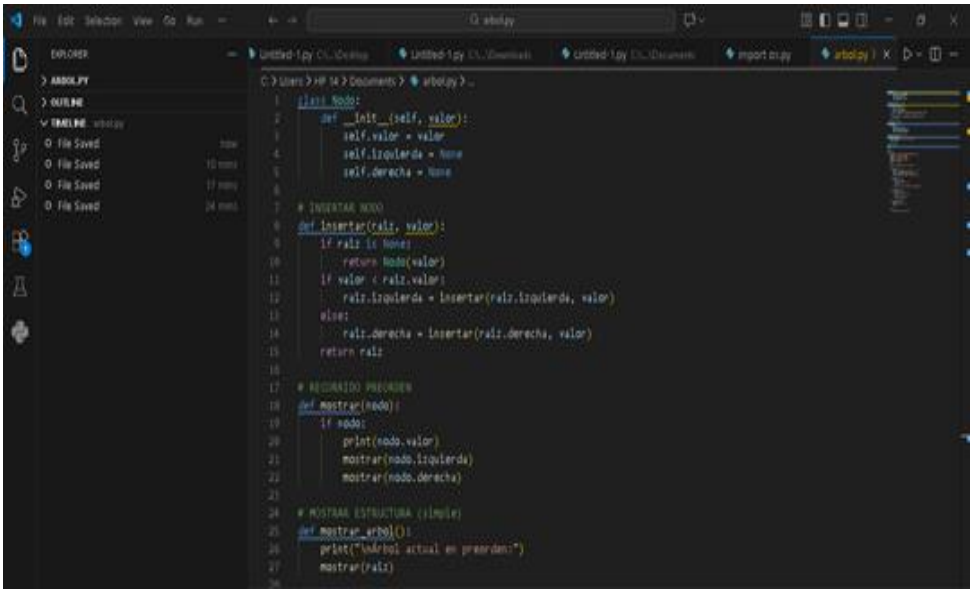
Análisis de la Estructura (Ventajas y Desventajas)

El uso de árboles binarios presenta varias ventajas, entre las cuales destaca la

organización jerárquica de los datos, lo que permite realizar búsquedas y recorridos de manera eficiente. Además, facilita la representación de estructuras complejas como expresiones matemáticas, sistemas de archivos y jerarquías organizacionales.

Sin embargo, también presenta desventajas, como la posibilidad de que el árbol se desbalancee, lo que afecta el rendimiento de las operaciones. Asimismo, su implementación es más compleja en comparación con estructuras lineales y requiere mayor uso de memoria debido a las referencias entre nodos

3. Imagen del Código



```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.izquierda = None
        self.derecha = None

# INSERTAR NODO
def insertar(raiz, valor):
    if raiz is None:
        return Nodo(valor)
    if valor < raiz.valor:
        raiz.izquierda = insertar(raiz.izquierda, valor)
    else:
        raiz.derecha = insertar(raiz.derecha, valor)
    return raiz

# RECORRIDO PREORDEN
def mostrar(nodo):
    if nodo:
        print(nodo.valor)
        mostrar(nodo.izquierda)
        mostrar(nodo.derecha)

# MOSTRAR ESTRUCTURA (simple)
def mostrar_arbol():
    print("\nÁrbol actual en preorden:")
    mostrar(raiz)
```

El presente código implementa una estructura de datos tipo árbol binario utilizando el lenguaje de programación Python. Para ello, se define una clase Nodo, la cual representa cada elemento del árbol y contiene como atributos un valor y dos referencias a sus nodos hijos: izquierdo y derecho. Esta representación permite modelar jerarquías de datos de forma eficiente mediante estructuras enlazadas.

Posteriormente, se construye manualmente un árbol binario asignando valores específicos a cada nodo y estableciendo las conexiones entre ellos, formando así una estructura jerárquica organizada. Este tipo de estructura es fundamental en aplicaciones como búsqueda, ordenamiento y representación de expresiones.

Además, se implementa una función recursiva denominada mostrar, la cual realiza un recorrido en preorden del árbol. Este tipo de recorrido visita primero la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho, lo que permite visualizar el contenido del árbol de manera estructurada. La salida del programa evidencia el correcto funcionamiento del algoritmo recursivo y la adecuada construcción del árbol binario.

Este ejercicio permite comprender los principios básicos de los árboles binarios, la manipulación de nodos enlazados y el uso de la recursividad para recorrer estructuras jerárquicas, constituyendo una base esencial en el estudio de estructuras de dato

4. Ejecución del Código

Durante la ejecución del programa se realiza la construcción de un árbol binario mediante la instanciación de objetos de la clase Nodo, donde cada nodo contiene un valor numérico y referencias a sus hijos izquierdo y derecho. Una vez definida la estructura, el sistema procede a ejecutar la función mostrar, la cual implementa un algoritmo recursivo para recorrer el árbol.

El recorrido aplicado es de tipo preorden, lo que implica que primero se procesa el nodo raíz, seguido del subárbol izquierdo y finalmente el subárbol derecho. Este comportamiento se evidencia en la salida del programa, donde los valores se imprimen en el siguiente orden: **10, 5, 3, 7, 20, 15.**

La ejecución confirma que la recursividad se aplica correctamente, ya que la función se invoca a sí misma para cada nodo existente hasta alcanzar los nodos hoja, donde las referencias son nulas. En ese punto, la función retorna automáticamente, permitiendo continuar con el flujo del recorrido sin generar errores.

Además, la salida obtenida demuestra que la estructura del árbol fue construida de manera adecuada, respetando la organización jerárquica de los nodos. Este tipo de ejecución es fundamental para validar el correcto funcionamiento de estructuras dinámicas y algoritmos de recorrido en árboles binarios

En cuanto al espacio, el uso de recursividad implica un consumo adicional en la pila de llamadas, que en el peor de los casos puede ser proporcional a la altura del árbol, es decir O(h). Si el árbol está balanceado, la altura será logarítmica, pero en un árbol desbalanceado puede llegar a ser lineal.”

5. Menú del Programa – Árbol Binario

Visualización del Menú Principal del Sistema

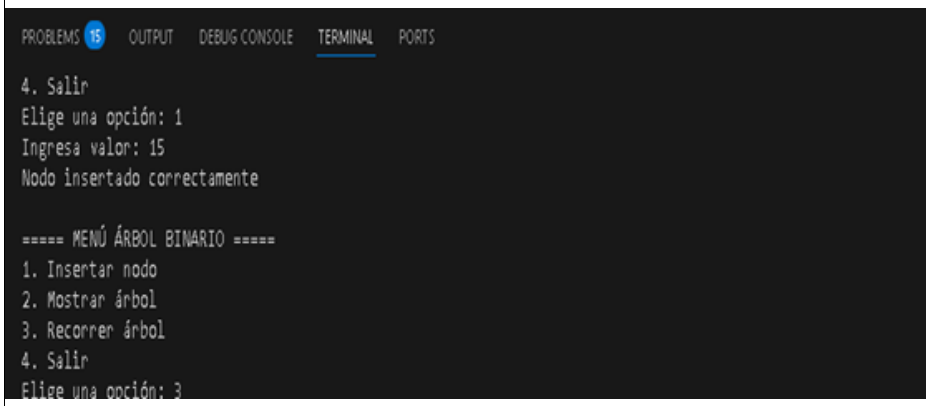
Descripción: La imagen muestra el panel de control principal del programa, diseñado para la gestión integral de estructuras de datos tipo árbol.

Opciones del Sistema: Se identifican las funcionalidades clave requeridas: inserción de nuevos nodos, visualización de la estructura y la ejecución de algoritmos de recorrido (preorden, inorden o postorden).

Control de Flujo: Se observa la validación de la opción 4 (Salir), la cual permite finalizar la ejecución del programa de manera controlada, asegurando el cierre adecuado de los procesos en el entorno de Python.

Entorno de Ejecución: La captura confirma que el software opera correctamente en una terminal de comandos, cumpliendo con el ambiente de escritorio solicitad

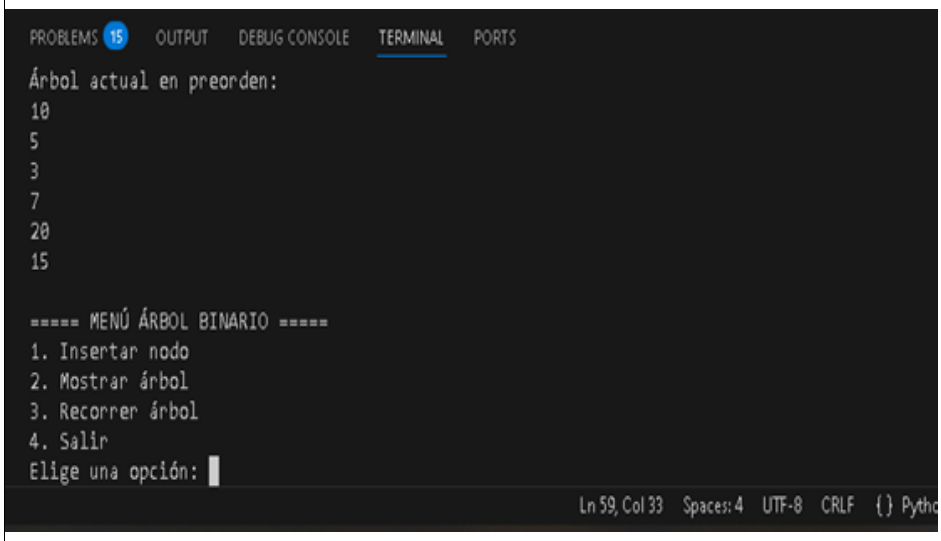
6. Reporteria



En esta etapa se procede a la elaboración de la reportería del práctico experimental, donde se documenta de manera clara y ordenada todo el proceso realizado durante el desarrollo de la actividad. Se incluye la descripción del problema planteado, el diseño de la solución mediante el uso de un árbol binario y la implementación del código en el lenguaje Python.

Asimismo, se presentan evidencias del funcionamiento del programa, tales como capturas de pantalla del código fuente y de su ejecución, donde se puede observar el recorrido del árbol en **preorden**. Estas evidencias permiten validar que la estructura fue correctamente construida y que el algoritmo recursivo funciona de manera adecuada.

7. Árbol actual en preorden



Dentro de la reportería también se detalla la lógica empleada en la creación de la clase Nodo, la forma en que se enlazan los nodos para formar el árbol y el funcionamiento de la función recursiva utilizada para el recorrido. Esto facilita la comprensión del comportamiento del programa y su correcta interpretación.

Finalmente, se incorporan análisis y conclusiones sobre los resultados obtenidos, destacando la importancia de las estructuras de datos tipo árbol en la programación, así como el uso de la recursividad como técnica fundamental para el procesamiento de estructuras jerárquicas. Todo esto se presenta siguiendo los lineamientos establecidos en la guía, garantizando un informe técnico claro, coherente y completo

Uso de Inteligencia Artificial

Para el desarrollo de este práctico se utilizó el agente de inteligencia artificial ChatGPT como herramienta de apoyo en la generación de código y redacción del informe. Se estima que aproximadamente el 40% del contenido fue asistido

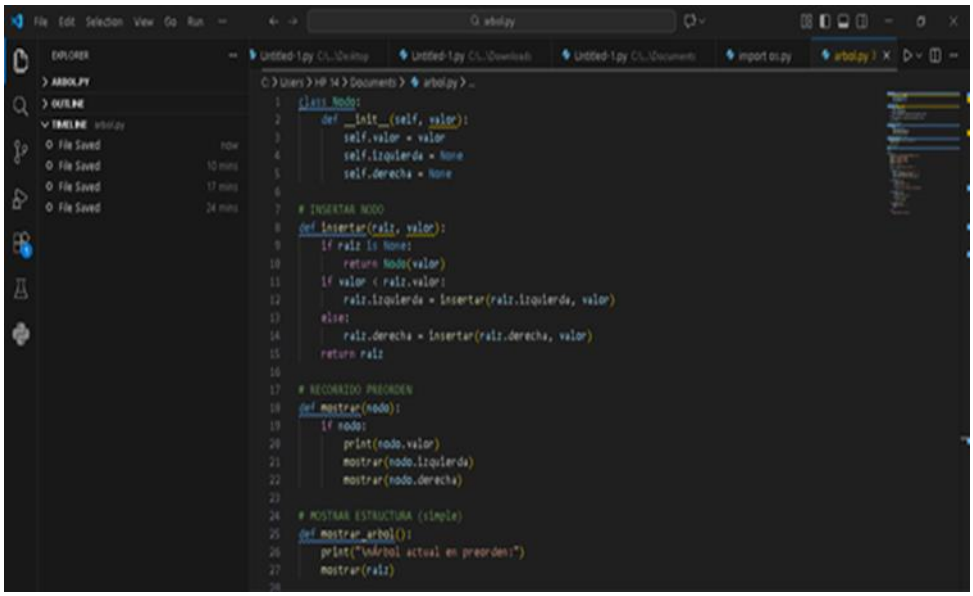
	por dicha herramienta, mientras que el resto corresponde a la implementación, análisis y adaptación realizada por el estudiante -
Principales resultados	Durante la ejecución del programa se obtuvo como resultado la impresión ordenada de los nodos del árbol binario siguiendo el recorrido en preorden. Los valores mostrados evidencian que la estructura fue correctamente construida y que el algoritmo recursivo funciona de manera adecuada. Los resultados permiten validar que la recursividad se ejecuta correctamente, ya que la función se llama a sí misma hasta recorrer todos los nodos del árbol, retornando sin errores al alcanzar nodos hoja. Se comprobó que la implementación en Python es eficiente y adecuada para este tipo de estructuras, facilitando la comprensión y visualización del comportamiento del árbol binario. Asimismo, las imágenes de los árboles permiten visualizar claramente la organización jerárquica de los datos, facilitando la comprensión del comportamiento de la estructura implementad
Conclusiones	El desarrollo de este práctico permitió comprender de manera clara el funcionamiento de los árboles binarios y su importancia en el ámbito de la programación. La implementación en Python facilitó la creación de la estructura y la aplicación de algoritmos recursivos.

Se evidenció que la recursividad es una herramienta fundamental para el recorrido de estructuras jerárquicas, permitiendo simplificar la lógica del programa y mejorar su eficiencia.

Finalmente, se concluye que el dominio de estructuras de datos como los árboles binarios es esencial para el desarrollo de soluciones eficientes en problemas reales, constituyendo una base sólida en la formación académica del estudiante.

Enlace: https://github.com/CJCuevas1/Practico-Experimental-Semana-16

1. Imagen del Código



2 . Árbol 1

```
    A
   / \
  B   C
 / \   \
D   E   F
```

3 . Árbol 2

```
      10
     /  \
    5    20
   / \   /
  3   7 15
```

4. Ejecución del Código

5. Menú del Programa – Árbol Binario

```
===== MENÚ ÁRBOL BINARIO =====
1. Insertar nodo
2. Mostrar árbol
3. Recorrer árbol
4. Salir
Elige una opción:
4. Salir
4. Salir
4. Salir
4. Salir
4. Salir
Elige una opción:
4. Salir
```

6. Reporteria

```
PROBLEMS 15   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

4. Salir
Elige una opción: 1
Ingresa valor: 15
Nodo insertado correctamente

===== MENÚ ÁRBOL BINARIO =====
1. Insertar nodo
2. Mostrar árbol
3. Recorrer árbol
4. Salir
Elige una opción: 3
```

7. Actual Árbol en preorden

```
PROBLEMS 15   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Árbol actual en preorden:
10
5
3
7
20
15

===== MENÚ ÁRBOL BINARIO =====
1. Insertar nodo
2. Mostrar árbol
3. Recorrer árbol
4. Salir
Elige una opción:
```

Bibliografías	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. Sedgewick, R., & Wayne, K. (2021). Algorithms (4th ed.). Addison-Wesley. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2022). Data Structures and Algorithms in Python (2nd ed.). Wiley. Zelle, J. M. (2021). Python Programming: An Introduction to Computer Science (3rd ed.). Franklin, Beedle & Associates.